

08 | Tekton：探索IaC与GitOps的另一条路径

潘野 · 云原生基础架构实战课



你好，我是潘野。

上一讲，我们学习了如何使用 GitHub Action 来管理 Terraform 代码，并且也实现了 GitOps 的整个过程。

虽然这种方式比较容易上手，但是 GitHub Action 存在一定的局限性。比如有些公司并没有选择 GitHub 作为代码管理平台，而是选择 Gitlab 作为公司的代码管理平台。或者是有些公司购买的是 Github Enterprise 版本，这时候有一些 GitHub 上的第三方插件就无法在内网环境里使用。针对这些场景，我们就需要找到一个适用度更好的 CI/CD 工具。

在持续集成领域，有很多持续集成、持续部署的工具，除了前一讲使用的 Github Action，还有大名鼎鼎的 Jenkins，来自 Kubernetes 社区的 Prow 等等，而今天我们将会选择使用 Tekton 来作为我们 CI/CD 的工具链。

为什么选择使用 Tekton?

和其他的 CI/CD 工具相比，Tekton 有三个优势。

第一，Tekton 的设计考虑到了多种使用场景，定制程度高且相对容易上手，能够适应各种复杂的需求。例如，它用 Task 这个资源来描述每个步骤细节，用 Pipeline 这个资源将各个步骤串联在一起。因此，相比 Jenkins Pipeline 中将步骤与流水线混合编写，Tekton 配置代码的可读性和可维护性会更好。

第二，Tekton 的可扩展性好。Tekton Catalog 是 Tekton 社区驱动的存储库，其中包含大量预制组件，可以直接使用这些组件，快速创建新的 CI/CD 流水线，或者扩展现有流水线；同时 Tekton 本身也支持扩展，我们也可以开发自定义的 Task 和 Pipeline 来满足特定的需求。

第三，Tekton 诞生自 Knative 项目，原生于 Kubernetes 平台，它与 Kubernetes 紧密结合，能够充分利用 Kubernetes 的动态扩展能力。

正是因为这三方面的优势，借助 Tekton，你可以**更容易地创建复杂的 CI/CD 工作流**，提高 GitOps 的定制能力，降低 GitOps 的维护压力。

Tekton 基本概念

不难发现，Tekton 是一个强大的 CI/CD 工具。我们接下来学习一下 Tekton 的基本概念，最重要的四个概念分别是 Task、TaskRun、Pipeline 和 PipelineRun。

Task

Task 中定义了完成特定任务所需的步骤和材料。它就好比烹饪食谱一样，包含多个步骤，每个步骤可以执行一个具体的命令或操作。例如一个 Task 可以包含以下步骤：

从代码仓库拉取代码

运行单元测试

构建镜像

推送镜像到镜像仓库

TaskRun

TaskRun 就是 Task 的实际执行，就像根据食谱烹饪一道菜一样，TaskRun 代表了一次具体的任务执行过程。每个 TaskRun 会创建一个 Pod，其中每个步骤对应 Pod 中的一个容器，TaskRun 会记录任务执行的状态，例如成功、失败或正在运行。

Pipeline

Pipeline 表示流水线模版，它就像一个工作流程图，定义了多个 Task 的执行顺序，最终合成一个完整的 CI/CD 流水线。例如，一个应用从构建到上线的 Pipeline 可以包含以下步骤：

构建代码

运行单元测试

构建镜像

推送镜像

部署应用程序

PipelineRun

PipelineRun 是 Pipeline 的实际执行，就像我们根据工作流程图完成一项工作一样，PipelineRun 代表了一次具体的流水线执行过程。每个 PipelineRun 会生成一条流水线记录，其中包含所有任务的执行状态。

如何将 GitOps 与 Tekton 结合使用？

回想下我们执行 terraform 的三个步骤，分别是 terraform init、terraform plan 和 terraform apply。

其实这些步骤，我们都可以通过定义 Tekton 的 Task 来完成。因为从 init 到 plan，再到 apply 这三步存在先后顺序，这是一个流水线的形态。对应到 Tekton 中的就是 Pipeline，所以这个 Pipeline 的全貌应该分成 4 个 Task。

1. 第一个 Task 是克隆你的 Terraform 代码的 Git 仓库。
2. 接下来，第二个 Task 是运行 `terraform init` 来初始化你的 Terraform 工作区。请注意，这时候不要忘记将 Terraform 的状态文件存进远端存储中。
3. 第三个 Task 是运行 `terraform plan` 来创建一个执行计划。
4. 最后一个 Task 是运行 `terraform apply` 来应用你的执行计划。

Trigger 类型及原理

我们定义好 Task 和 Pipeline 之后，需要触发 Task 或者 Pipeline 的时候，必须手动创建一个 TaskRun 或者 PipelineRun，这样显然不符合 GitOps 自动化的要求。

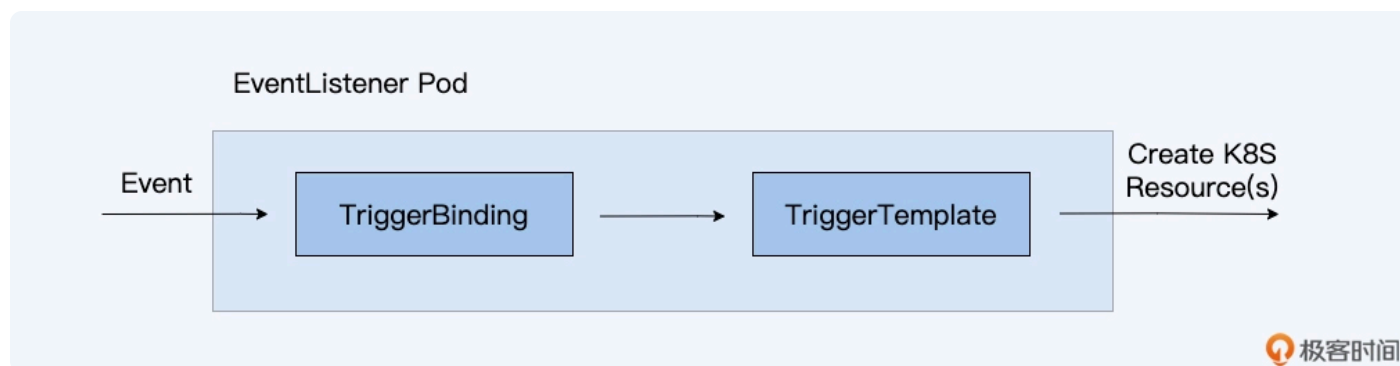
这时候，我们就需要用到 Tekton Trigger 组件，它是专门用来解决 CI/CD 流水线触发问题的。它可以监听各种来源的事件，并根据事件信息自动触发 Task 或 Pipeline 的执行，从而自动化我们的 CI/CD 工作流。Tekton 支持多种类型的 Trigger，具体包括这几类。

Git Trigger：监听 Git 仓库的更改。

Event Trigger：监听 Kubernetes 事件。

Time Trigger：定期触发。

你可以从 [官方的代码仓库](#) 看到 Tekton 支持的各类 Trigger 的使用样例。接下来，我们看看 Trigger 是如何工作的，流程图我贴在了后面。



Tekton EventListener 用来监听外部事件，当事件进入到 EventListener 内部，TriggerBinding 就会从事件内容中提取对应参数，然后将参数传递给 TriggerTemplate。

TriggerTemplate 则会根据预先定义的模版以及收到的参数，创建 TaskRun 或者 PipelineRun 对象。

配置环节

明确了思路以后，我们动手尝试一下，用 Tekton 创建一个能执行 Terraform 操作的流水线。

首先，你需要在你的 Kubernetes 集群中安装和设置 Tekton。这里我们需要安装 Tekton 的 Pipeline 和 Triggers 组件。

因为安装部署只需要 apply tekton 配置文件即可，所以这里我贴出安装 Tekton 的命令。至于更细节的内容，你可以参考官方文档中 [Installation 的这个章节](#)。

Tekton Pipeline 和 Trigger 的安装命令如下：

 复制代码

```
1 kubectl apply --filename https://storage.googleapis.com/tekton-releases/pipeline/
2
3 kubectl apply --filename \
4 https://storage.googleapis.com/tekton-releases/triggers/latest/release.yaml
5
6 kubectl apply --filename \
7 https://storage.googleapis.com/tekton-releases/triggers/latest/interceptors.yaml
```

同时，我们也把 Tekton Dashboard 安装上，方便我们使用过程中排查流水线的运行状态和问题。安装的命令如下：

 复制代码

```
1 kubectl apply --filename https://storage.googleapis.com/tekton-releases/dashboard
```

Github Webhook 的配置

Webhook 的配置分为两个部分，第一部分是 Trigger 的配置，第二部分是 Github 状态的配置。

我们先讲 Trigger 部分的配置，这部分安装配置比较简单，我将所需要的文件和配置的方法放在了 [📁 代码仓库](#) 里供你参考。下面，我简单描述下几个文件的作用。

首先来看 [📄 github-eventlistener-interceptor.yaml](#) 这个文件。

当我们提交源代码到 GitHub 的时候，需要触发 Tekton 的任务运行。所以首先需要完成这个触发器。这里就可以通过 EventListener 这个资源对象来完成。

注意，对于 resources 中的 ServiceType，你需要根据实际情况选择。如果你在云上，选择 LoadBalancer；如果你是在自建机房内，没有负载均衡器条件的，就可以选择 NodePort。

 复制代码

```
1 resources:
2   kubernetesResource:
3     serviceType: LoadBalancer
```

这样就生成了 EventListener 的访问地址。当 Service 生成之后，我们还要将 tk.github.51.cafe 的 DNS 解析到 Service 的 IP 上，方便接下来的配置。

 复制代码

```
1 > kubectl get svc -n terraform-tekton el-github-listener
2 NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)
3 el-github-listener                 LoadBalancer        10.0.138.249    4.189.196.54     8080:31669/TCP,
```

然后我们来配置 Github Webhook，配置如下图所示，这里选择什么情况下触发 Webhook，如果你不熟悉 Github Event，这里可以选择“Send me everything”。如果你对 Github Event 比较了解，可以选择“individual event”，根据你的需求来灵活定制。

Webhooks / Manage webhook

Settings

Recent Deliveries

We'll send a POST request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in [our developer documentation](#).

Payload URL *

http://tk.github.51.cafe:8080

Content type

application/json

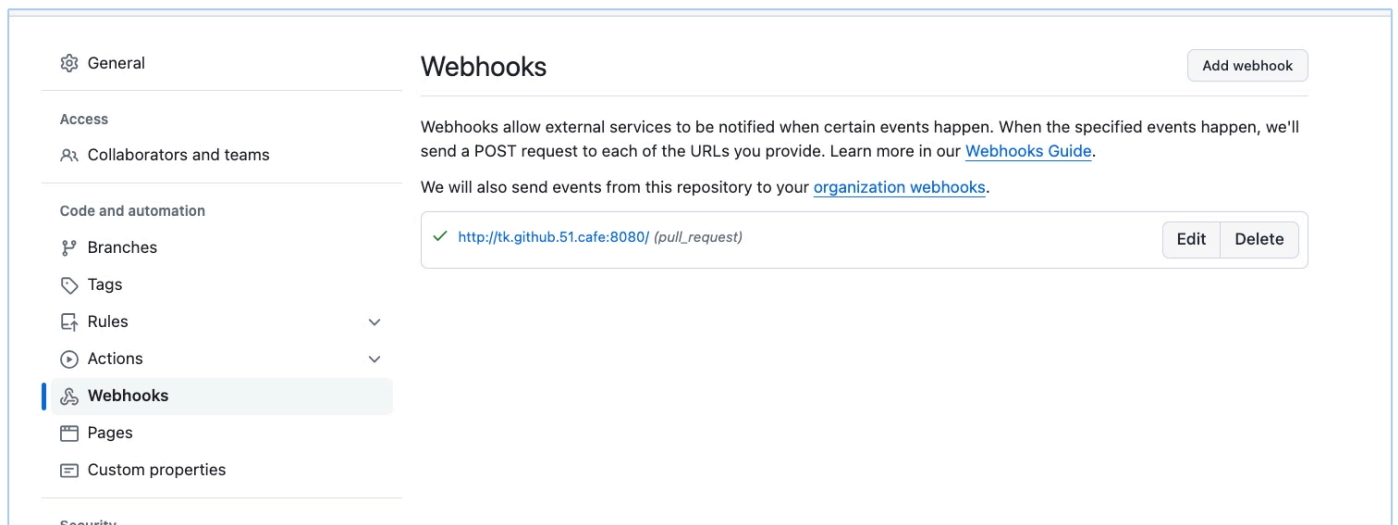
Secret

If you've lost or forgotten this secret, you can change it, but be aware that any integrations using this secret will need to be updated. — [Change Secret](#)

Which events would you like to trigger this webhook?

- ☐ Just the push event.
- ☐ Send me **everything**.
- ☒ Let me select individual events.

一切配置好了之后，回到 Webhook 页面，此时你应该看到一个绿勾，这代表你的 Webhook 是可以正常工作了。



配置好 Webhook 之后，我们再来讲讲 GitHub 状态的配置。配置 GitHub PR 状态的目的是方便你快速定位到这个 PR 所对应的 Tekton 流水线，因为 Tekton 官方已经提供了插件，我们只要根据 [🔗 插件文档](#) 安装和 [🔗 配置](#) 即可。

下图展示了 Pull Request 的 CI 状态。

add trigger config files #6

 Open pye99 wants to merge 1 commit into `main` from `trigger` 

 Conversation 0

 Commits 1

 Checks 0

 Files changed 4



pye99 commented 21 minutes ago

Member ...

No description provided.



  add trigger config files

70f9966

Add more commits by pushing to the `trigger` branch on `cloudnative-automation/tekton-terraform`.



 Some checks haven't completed yet

[Hide all checks](#)

1 pending check



`continuous-integration/tekton` Pending — Build has started

[Details](#)



This branch has no conflicts with the base branch

Merging can be performed automatically.

Merge pull request



You can also [open this in GitHub Desktop](#) or view [command line instructions](#).



Add a comment

 极客时间

Task 定义

接下来，我们开始定义我们的 Task。

Tekton 官方提供了很多关于 Git 和 Terraform 操作的 Task，我们可以直接拿来使用。具体你可以从 [官网的插件仓库](#) 中获取到你想要的 Task 插件。

这里我们需要用到这两个 Task 插件。[git-clone](#) 插件能够帮助我们把代码克隆到 Tekton 的容器中。[terraform-cli](#) 则是我们运行 Terraform 命令的基础。

同样，具体安装方法你可以参考插件的官方文档。

Git

首先是定义 Git 方面的操作，这里我们重点关注核心的配置部分。完整的配置文件请参考 [🔗 GitHub 仓库中的配置](#)。

首先，我们要在 steps 中定义 git clone 操作。

 复制代码

```
1 apiVersion: tekton.dev/v1beta1
2 kind: Task
3 metadata:
4   name: git-clone
5 spec:
6   workspaces:
7     (省略..)
8   params:
9     - name: url
10      type: string
11     - name: gitInitImage
12      type: string
13     (...)
14   steps:
15     - name: clone
16       image: "$(params.gitInitImage)"
17       env:
18         - name: HOME
19           value: "$(params.userHome)"
20         - name: PARAM_URL
21           value: $(params.url)
```

可以看到，我们把 git clone 中需要的一些参数定义在了 `params` 中，例如 Git 仓库的 URL，clone 时用的 Token 等。

init

当我们执行 `terrafrom init` 这个动作的时候，其实主要目的是将相对应的模块下载下来，并且定义状态文件所需要的一些权限。这里我建议你在 Task 中用 script 字段配合 params 来实现初始化。

这里肯定有同学会问了，AWS_SECRET_ACCESS_KEY 和 AWS_ACCESS_KEY_ID 是从哪里读取到的？

这里是我们预先将 AWS_SECRET_ACCESS_KEY 和 AWS_ACCESS_KEY_ID 以 Secret 的形式存在了 Tekton 的 namespace 中，然后以环境变量的形式导入 Tekton。

 复制代码

```
1 - name: init
2   image: $(params.image)
3   workingDir: $(workspaces.source.path)
4   env:
5     - name: AWS_SECRET_ACCESS_KEY
6       valueFrom:
7         secretKeyRef:
8           name: aws-iam-key
9           key: $(params.iam-key)
10    - name: AWS_ACCESS_KEY_ID
11      valueFrom:
12        secretKeyRef:
13          name: aws-iam-key
14          key: $(params.iam-id)
15    script: |
16      #!/usr/bin/env sh
17      set -eu
18      terraform init
```

plan

接下来是 terraform plan 这步。我们将 Terraform 的执行步骤放在 terraform-cli 的 script 字段中，script 字段中，你可以嵌入任何你熟悉的脚本语言来运行 Terraform 命令。

 复制代码

```
1 - name: terraform-cli
2   image: $(params.image)
3   workingDir: $(workspaces.source.path)/$(params.reponame)
4   script: |
5     #!/usr/bin/env sh
6     set -eu
7     TERRAFORM_VERSION=`grep -A1 "terraform_version" variables.tf |grep "default"
8     if [ ! -z "$TERRAFORM_VERSION" ] ; then
```

```

9     export TFENV_TERRAFORM_VERSION=$TERRAFORM_VERSION
10 fi
11 # terraform --version
12 tfenv list
13 terraform plan -input=false -no-color -out=tfplan
14 terraform show tfplan -no-color 2>&1 | tee tfplan.txt

```

terraform apply

之后是创建执行计划。在 script 中，我们定义了执行 `terraform apply --auto-approval` 这条命令，同时将输出结果写入一个文本文件中，这样的作用是通过其他的插件将 apply 的输出内容回传到 Github 上。

 复制代码

```

1 - name: terraform-cli
2   image: $(params.image)
3   workingDir: $(workspaces.source.path)/$(params.reponame)
4   script: |
5     #!/usr/bin/env sh
6     terraform apply -input=false -auto-approve -no-color 2>&1 | tee tfapply.txt

```

Pipeline 配置

定义完 Task 之后，我们需要将这些 Task 串联成一个 Pipeline。同样为了让你抓住重点，我将配置文件的核心部分提炼出来。完整的配置文件请参考 [🔗 GitHub 仓库中的配置](#)。

 复制代码

```

1 apiVersion: tekton.dev/v1beta1
2 kind: Pipeline
3 metadata:
4   name: terraform-pr
5 spec:
6   tasks:
7     - name: fetch-from-git
8       taskRef:
9         name: git-clone
10    - name: terraform-plan
11      runAfter: [fetch-from-git]
12    - name: git-commit-terraform-results
13      runAfter: [terraform-plan]

```

```
14 - name: terraform-apply
15     runAfter: [git-commit-terraform-results]
16 finally:
17 - name: report-pipeline-status-to-github
18     taskRef:
19         name: github-set-status
```

对照代码我们看看串联思路是怎样的。在 Tasks 中，我们将之前定义的每个小 Task 放在里面，同时用 runAfter 这个关键词来定义 Task 的执行顺序。然后用 finally 这个关键词定义了一个收尾的 Task，将之前运行的所有结果送回到 GitHub 的页面上。

这样我们就完成了一个基于 Tekton 的 GitOps 流水线！

Github Action 与 Tekton 的比较

我们完成了两种不同的 CI/CD 工具的 GitOps，这里我们做一个比较。

相比 Tekton 来说，GitHub Actions 使用起来更加简单直接。我们只需要定义好 workflow 的配置，并将 AWS 的相关密钥存储在 Github 仓库的 Secrets and variable 中，就基本完成了配置。对于 GitHub 项目，使用 GitHub Actions 进行 CI/CD 流程可以说是无缝集成，设置和使用都非常便捷。

但是 Github 与云服务的集成可能不够完美，比如 runner 需要一个单独的机器来运行，也需要更多的配置工作，这使得 runner 弹性扩展能力稍显不足；再比如它目前还缺少一些高级功能，如审计追踪等；另外，因为 GitHub Action 是 Github 的服务，所以在 GitHub 以外的代码托管平台上无法使用。

至于 Tekton 呢，我们可以将一个任务拆解成小任务，通过 Pipeline 将这些小任务串联起来，定制化程度很高，并且与各个云提供商的集成也比较容易。其次，Tekton 是云原生的 CI/CD 工具，所以与 Kubernetes 配合度好，在高度定制化的基础上又具有弹性扩展的能力。

然而，Tekton 也存在一些不足。例如，虽然它的上手难度不大，但是配置文件比较冗长，新手可能会望而生怯。此外，尽管其高度定制化的特性为复杂的工作流提供了可能，但这同时也

意味着我们需要花费更多的时间和精力来配置和管理。这与一些更加“开箱即用”的 CI/CD 工具形成了对比。

我将 GitHub Action 与 Tekton 的对比做成了一个表格，供你参考。

	Github Actions	Tekton
平台	GitHub	Kubernetes
部署方式	托管/自托管	自托管
易用性	易于使用，提供直观的 UI	有一定学习曲线
灵活性	灵活，支持多种语言和框架	更加灵活，可扩展性强
社区支持	庞大的社区，丰富的资源	社区相对较小



总的来说，选择哪种工具取决于具体的项目需求和团队的偏好。你可以考虑综合前面的对比项，选出更适合自己的团队的工具。

总结

今天，我们通过分解运行 Terraform 代码的三大步骤 init、plan、apply，一起学习了如何使用 Tekton。我们在掌握了一种新的 CI/CD 工具同时，也学会了如何使用 Tekton 来管理我们的 Terraform 代码，实现了基于另外一种工具的 GitOps。

Tekton 流水线配置中还有大量的配置细节与注释，完整配置超过 1000 行，因此课程里着重和你分享了配置重点。完整配置我放在了代码仓库中，请你课后一定要仔细研读代码与注释，这样才能完整地理解基于 Tekton 的 GitOps 的实现方式。

最后，我带你比较了 Github Action 与 Tekton，分析了两者的优势和不足。虽然我只是对比了两款 CI/CD 工具，但你不必局限于此，完全可以参考我的分析对比思路，想想有什么其他方式也能自动化地实现 Terraform 的执行过程（毕竟工具可能更新换代，但都是为了自动完成 Terraform 的执行过程）。

至此我们已经打下了基于 GitOps 的 IaC 的基础。但是我们 GitOps 之旅还未走完，离最后真正的交付还有最后的一公里，下一讲我将带你继续探索，敬请期待。

思考题

在 terraform init 的这个环节中，我们加载模块代码往往需要花费 5 到 10 分钟。你有哪些手法可以加速加载模块代码么？

欢迎在评论区与我讨论。如果这一讲对你有启发，也欢迎分享给身边更多朋友。

AI智能总结

- 1. Tekton是一个CI/CD工具，具有设计灵活、可扩展性好、与Kubernetes紧密结合的优势。
- 2. Tekton的基本概念包括Task、TaskRun、Pipeline和PipelineRun，用于定义任务步骤、执行任务、定义流水线模版和执行流水线。
- 3. 结合GitOps与Tekton使用，可以通过定义Task来完成Terraform操作，然后使用Tekton Trigger组件实现自动化CI/CD工作流。
- 4. Tekton Trigger组件支持多种类型的触发器，包括Git Trigger、Event Trigger和Time Trigger。
- 5. 在配置环节中，需要安装和设置Tekton的Pipeline和Triggers组件，并配置Github Webhook以及GitHub PR 状态。
- 6. Tekton官方提供了很多关于Git和Terraform操作的Task插件，可以直接使用。
- 7. GitHub Actions使用起来更加简单直接，而Tekton具有高度定制化和与Kubernetes配合度好的特点，选择取决于具体项目需求和团队偏好。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (3)

最新 精选



alex run
2024-04-21 来自上海

解决tf init加载问题，是不是可以用pvc缓存起来

作者回复: Hi Alex

是的，这是其中之一的方法，你也可以把modules下载回来，放在一些本地的http服务器上，减少外网传输时间。



橙汁

2024-04-10 来自北京

tekton有点之前用jenkins那味了，又是插件又是定义变量 最后总结的相当可以，之前看了kubernetes的gitops形式 其实gitops并不局限 iac 自动化 集群等任何方面都能用gitops方式

作者回复: Hi，橙汁你好

是的，在现在的基础设施管理工作中，GitOps模式是不可缺少的一环了。也不仅局限于IaC，集群管理，甚至一些场景下的用户管理都可以套用gitops模式。



Geek_45a572

2024-04-10 来自陕西

可以使用cache.将provider缓存起来

